

CARRERA DE INGENIERÍA DE SOFTWARE

EXAMEN PRÁCTICO — UNIDAD 1

Generalidades de los Sistemas Distribuidos

Datos generales

Asignatura	Aplicaciones Distribuidas (ISR-701)
Unidad	1 — Generalidades de los Sistemas Distribuidos
Tipo de evaluación	Examen práctico de unidad — Individual
Modalidad	Laboratorio con defensa técnica oral
Duración	2 horas de laboratorio
Período académico	Regular 2025-2026 SPA
Código del instrumento	EV-PR-U1 (ajustable según planificación)
Docente	ING. GUERRERO ULLOA GLEISTON CICERON

Estudiante	VINUEZA SANCHEZ HAROLD NICOLAS
Fecha	26/06/2026
URL del repositorio	https://github.com/Harold-Vinueza/APLICACIONES-DISTRIBUIDAS/tree/master/Prueba
Calificación (sobre 10)	_____

1. Competencia y resultado de aprendizaje evaluados

Este examen valora la capacidad del estudiante para analizar las características y principios de los sistemas distribuidos, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica, y para implementar de forma funcional los mecanismos fundamentales de la unidad: comunicación entre procesos, sincronización mediante relojes lógicos, tolerancia a fallos, coordinación y seguridad básica.

2. Planteamiento del problema

Una empresa de logística opera un servicio de registro de operaciones llamado RegistroDistribuido. El servicio se compone de tres nodos réplica (N1, N2, N3) que reciben “operaciones” (por ejemplo, registros de movimientos) provenientes de uno o varios clientes. Por razones de disponibilidad, los nodos deben coordinarse entre sí, mantener un orden coherente de los eventos que registran y seguir funcionando aunque alguno de ellos deje de responder.

Su tarea es construir un prototipo funcional de este servicio que demuestre, sobre un caso pequeño y ejecutable, los principios estudiados en la Unidad 1. No se evalúa la escala ni la persistencia en disco, sino la correcta aplicación de los mecanismos distribuidos. El sistema puede ejecutarse como tres procesos en la misma máquina (distintos puertos) o como tres contenedores.

3. Requerimientos a implementar

Parte A — Comunicación entre procesos (peso orientativo 20%)

- Implemente la comunicación cliente–nodo (y nodo–nodo) usando sockets TCP o gRPC.

- Delimite correctamente los mensajes (longitud previa, separador o framing) de modo que dos operaciones enviadas seguidas no se solapen ni se trunquen en el receptor.
- Un cliente debe poder enviar una operación y recibir la confirmación del nodo.

Cliente.java

```
package ec.edu.uteq.distribuidas.client;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.Socket;

public class Client {

    public static void main(String[] args) {

        try {

            System.out.println("Conectando al nodo...");

            Socket socket = new Socket("localhost", 5001);

            System.out.println("Conexión establecida.");

            DataOutputStream out =
                new DataOutputStream(socket.getOutputStream());

            DataInputStream in =
                new DataInputStream(socket.getInputStream());

            //String operation = "Movimiento_001";
            String operation = "TOKEN123|Movimiento_001";
            //String operation = "TOKEN_MALO|Movimiento_001";

            System.out.println("Enviando operación: " + operation);

            out.writeUTF(operation);
            out.flush();

            System.out.println("Esperando respuesta del nodo...");

            String response = in.readUTF();

            System.out.println("Respuesta del nodo: " + response);

            in.close();
            out.close();
            socket.close();

        } catch (IOException e) {

            System.err.println("Error en el cliente:");
            e.printStackTrace();

        }

    }

}
```

Esta clase establece una conexión TCP con un nodo del sistema distribuido para enviar una solicitud

acompañada de un token de validación. Posteriormente, recibe y presenta la respuesta generada por el nodo tras procesar la operación solicitada.

Node.java

```
package ec.edu.uteq.distribuidas.core;

import ec.edu.uteq.distribuidas.consensus.BullyElection;
import ec.edu.uteq.distribuidas.consensus.Coordinator;
import ec.edu.uteq.distribuidas.model.NodeInfo;
import ec.edu.uteq.distribuidas.model.NodeStatus;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.HashMap;
import java.util.Map;

public class Node {

    private final int nodeId;
    private final int port;
    private final LamportClock clock;
    private final NodeInfo[] peers;

    private final Map<Integer, NodeStatus> nodeStatusMap =
        new HashMap<>();

    public Node(int nodeId, int port) {

        this.nodeId = nodeId;
        this.port = port;
        this.clock = new LamportClock();

        this.peers = new NodeInfo[]{
            new NodeInfo(1, 5001),
            new NodeInfo(2, 5002),
            new NodeInfo(3, 5003)
        };

        for (NodeInfo peer : peers) {
            if (peer.getNodeId() != nodeId) {
                nodeStatusMap.put(
                    peer.getNodeId(),
                    new NodeStatus()
                );
            }
        }

        Coordinator.setCoordinatorId(3);
    }

    public void start() {

        startHeartbeatSender();
        startFailureDetector();

        System.out.println(
            "Nodo N" + nodeId +
```

```

        " escuchando en puerto " + port
    );

    System.out.println(
        "Coordinador actual: N" +
        Coordinator.getCoordinatorId()
    );

    try (ServerSocket serverSocket = new ServerSocket(port)) {
        while (true) {
            Socket clientSocket = serverSocket.accept();

            try {
                DataInputStream in =
                    new DataInputStream(clientSocket.getInputStream());

                DataOutputStream out =
                    new
DataOutputStream(clientSocket.getOutputStream());

                String message = in.readUTF();

                /*
                 * HEARTBEAT
                 */
                if (message.startsWith("HEARTBEAT|")) {

                    int senderId =
                        Integer.parseInt(message.split("\\|")[1]);

                    NodeStatus status =
                        nodeStatusMap.get(senderId);

                    if (status != null) {
                        status.updateHeartbeat();
                        status.setActive(true);
                    }

                    out.writeUTF("HB_OK");
                    out.flush();
                    continue;
                }

                /*
                 * REPLICA
                 */
                if (message.startsWith("REPLICA|")) {

                    String[] parts = message.split("\\|");

                    int receivedLamport =
                        Integer.parseInt(parts[1]);

                    String operation = parts[2];

                    int newTime = clock.update(receivedLamport);

```

```

        System.out.println(
            "[Lamport=" + newTime + "] N" +
            nodeId + " recibió réplica: " +
            operation
        );

        out.writeUTF("REPLICA_OK");
        out.flush();
        continue;
    }

    /*
     * CLIENTE
     */
    String[] parts = message.split("\\|");

    if (parts.length != 2) {

        out.writeUTF("ERROR: FORMATO INVALIDO");
        out.flush();
        continue;
    }

    String token = parts[0];
    String operation = parts[1];

    if (!TokenValidator.validate(token)) {

        out.writeUTF("ERROR: TOKEN INVALIDO");
        out.flush();
        continue;
    }

    int timestamp = clock.increment();

    System.out.println(
        "[Lamport=" + timestamp + "] Nodo N" +
        nodeId + " recibió operación: " +
        operation
    );

    replicateOperation(operation);

    out.writeUTF("OK");
    out.flush();

    } finally {
        clientSocket.close();
    }
}

} catch (IOException e) {

    System.err.println("Error en nodo N" + nodeId);
    e.printStackTrace();
}

}

private void replicateOperation(String operation) {

```

```

        for (NodeInfo peer : peers) {

            if (peer.getNodeId() == nodeId) continue;

            try {

                Socket socket =
                    new Socket("localhost", peer.getPort());

                DataOutputStream out =
                    new DataOutputStream(socket.getOutputStream());

                out.writeUTF(
                    "REPLICA|" +
                        clock.getTime() +
                        "|" +
                        operation
                );

                out.flush();
                socket.close();

            } catch (Exception e) {
                System.out.println("No se pudo replicar a N" +
peer.getNodeId());
            }
        }

        /*
         * HEARTBEATS
         */
        private void startHeartbeatSender() {

            Thread thread = new Thread(() -> {

                while (true) {

                    try {

                        for (NodeInfo peer : peers) {

                            if (peer.getNodeId() == nodeId) continue;

                            try {

                                Socket socket =
                                    new Socket("localhost", peer.getPort());

                                DataOutputStream out =
                                    new
DataOutputStream(socket.getOutputStream());

                                out.writeUTF("HEARTBEAT|" + nodeId);

                                out.flush();
                                socket.close();

                            } catch (Exception ignored) {}

                        }

                    }

```

```

        Thread.sleep(5000);
    } catch (Exception ignored) {}
    }

});

thread.setDaemon(true);
thread.start();
}

/*
 * FALLAS + BULLY
 */
private void startFailureDetector() {

    Thread thread = new Thread(() -> {

        while (true) {

            try {

                long now = System.currentTimeMillis();

                for (Map.Entry<Integer, NodeStatus> entry
                    : nodeStatusMap.entrySet()) {

                    NodeStatus status = entry.getValue();

                    long elapsed =
                        now - status.getLastHeartbeat();

                    if (elapsed > 10000 && status.isActive()) {

                        status.setActive(false);

                        System.out.println(
                            "Nodo N" + entry.getKey() +
                            " marcado como CAIDO"
                        );

                        if (entry.getKey() ==
                            Coordinator.getCoordinatorId()) {

                            System.out.println(
                                "Coordinador caído → iniciando elección
Bully..."
                            );

                            int newLeader =
                                BullyElection electLeader(
                                    nodeId,
                                    nodeStatusMap
                                );

                            Coordinator.setCoordinatorId(newLeader);

                            System.out.println(
                                "Nuevo coordinador: N" + newLeader

```

```

    });
    thread.setDaemon(true);
    thread.start();
}

```

Esta clase gestiona la recepción y procesamiento de solicitudes dentro del sistema distribuido, validando credenciales, registrando eventos mediante relojes de Lamport y replicando información entre nodos. Adicionalmente, supervisa el estado de los demás nodos mediante heartbeats y aplica el algoritmo Bully para mantener la coordinación ante posibles fallos.

Evidencias:

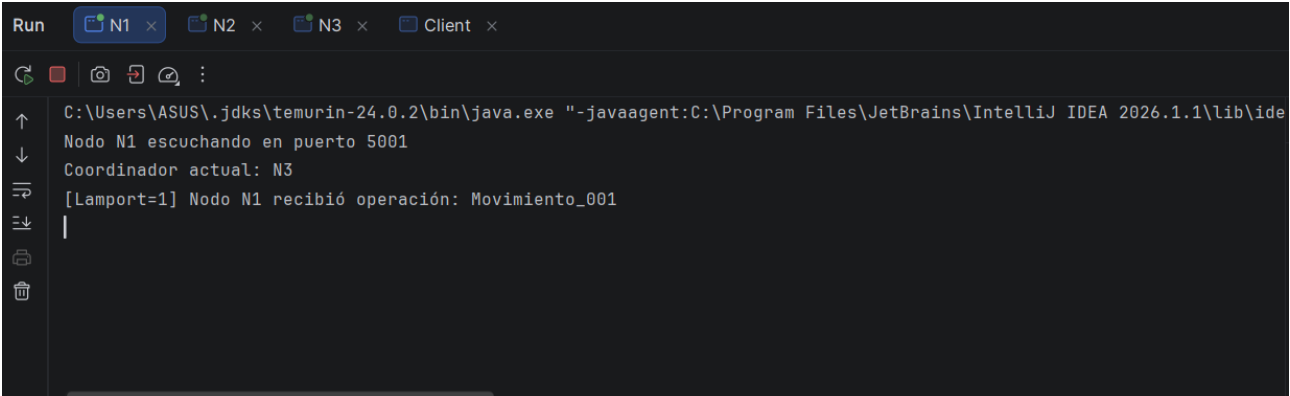


```

Run N1 x N2 x N3 x Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Conectando al nodo...
Conexión establecida.
Enviando operación: TOKEN123|Movimiento_001
Esperando respuesta del nodo...
Respuesta del nodo: OK
Process finished with exit code 0

```

Imagen 1. Se evidencia el establecimiento de la conexión con el nodo y el envío de una operación autenticada. Posteriormente, se recibe la confirmación de que la solicitud fue procesada de manera satisfactoria.



```

Run N1 x N2 x N3 x Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N1 escuchando en puerto 5001
Coordinador actual: N3
[Lamport=1] Nodo N1 recibió operación: Movimiento_001

```

Imagen 2. Se observa la recepción de la operación enviada por el cliente y su respectivo registro mediante el reloj lógico de Lamport, permitiendo mantener un orden coherente de los eventos.


```

Run  N1 x  N2 x  N3 x  Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N2 escuchando en puerto 5002
Coordinador actual: N3
[Lamport=2] N2 recibió réplica: Movimiento_001

```

Imagen 3. Se muestra la recepción de la operación replicada desde otro nodo, contribuyendo a la sincronización de la información y a la consistencia del sistema distribuido.

```

Run  N1 x  N2 x  N3 x  Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N3 escuchando en puerto 5003
Coordinador actual: N3
[Lamport=2] N3 recibió réplica: Movimiento_001

```

Imagen 4. Se muestra la recepción de la operación replicada desde otro nodo, contribuyendo a la sincronización de la información y a la consistencia del sistema distribuido.

Parte B — Relojes lógicos de Lamport (peso orientativo 20%)

- Cada nodo mantiene un reloj lógico de Lamport e implementa correctamente sus reglas: incremento ante un evento local y, al recibir un mensaje, fijar el reloj en $\max(\text{local}, \text{recibido}) + 1$.
- El sistema debe generar un registro (log) de eventos con un orden total consistente, usando el identificador de nodo como criterio de desempate ante marcas iguales.

LamportClock.java

```

package ec.edu.uteg.distribuidas.core;

public class LamportClock {

    private int time = 0;

    public synchronized int increment() {
        time++;
        return time;
    }

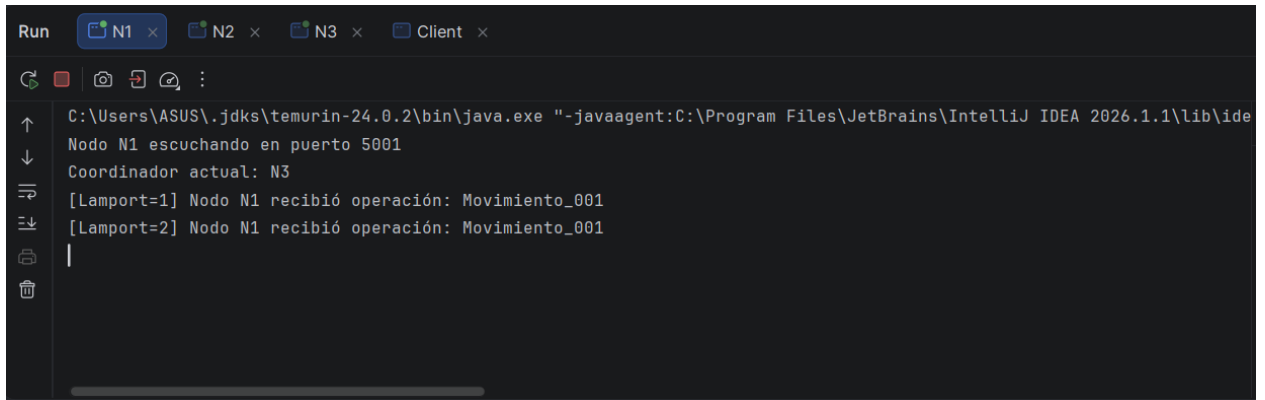
    public synchronized int update(int receivedTime) {
        time = Math.max(time, receivedTime) + 1;
        return time;
    }
}

```

```
public synchronized int getTime() {  
    return time;  
}
```

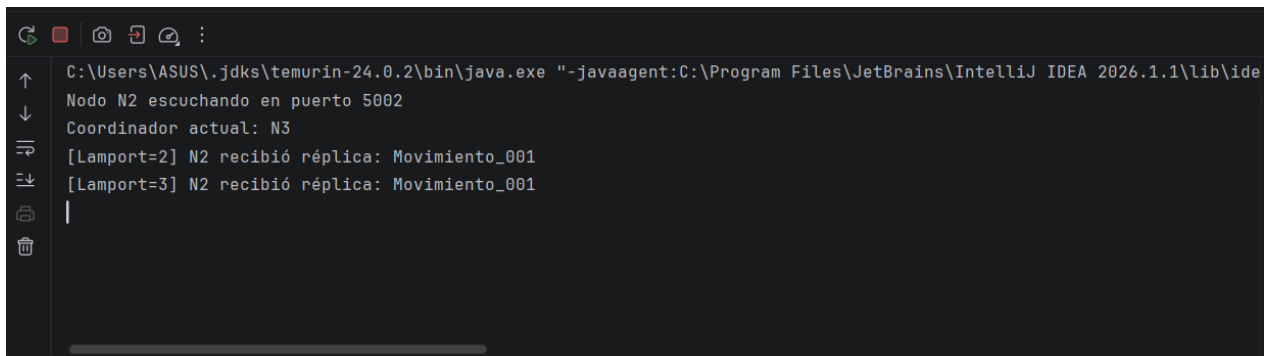
Esta clase implementa un reloj lógico de Lamport, permitiendo asignar marcas de tiempo a los eventos que ocurren en el sistema distribuido. En relación con Node.java, se utiliza para mantener el orden lógico de las operaciones recibidas y replicadas entre nodos, garantizando una secuencia coherente de eventos independientemente del tiempo físico de cada equipo.

Evidencias:



```
Run N1 x N2 x N3 x Client x  
C:\Users\ASUS\.jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide  
Nodo N1 escuchando en puerto 5001  
Coordinador actual: N3  
[Lamport=1] Nodo N1 recibió operación: Movimiento_001  
[Lamport=2] Nodo N1 recibió operación: Movimiento_001
```

Imagen 5. Registro de un evento local mediante el reloj lógico de Lamport. Se observa la asignación de una marca temporal lógica al momento de recibir una operación desde el cliente, permitiendo registrar el evento dentro de una secuencia ordenada.



```
C:\Users\ASUS\.jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide  
Nodo N2 escuchando en puerto 5002  
Coordinador actual: N3  
[Lamport=2] N2 recibió réplica: Movimiento_001  
[Lamport=3] N2 recibió réplica: Movimiento_001
```

Imagen 6. Actualización del reloj lógico tras la recepción de una réplica. Se evidencia la recepción de una operación proveniente de otro nodo y la correspondiente actualización del reloj lógico, contribuyendo a mantener la coherencia temporal entre los eventos.

```

Run  N1 x  N2 x  N3 x  Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N3 escuchando en puerto 5003
Coordinador actual: N3
[Lamport=2] N3 recibió réplica: Movimiento_001
[Lamport=3] N3 recibió réplica: Movimiento_001

```

Imagen 7. Registro distribuido de eventos en un nodo réplica. Se muestra el procesamiento de una operación replicada y su registro mediante una marca temporal lógica, favoreciendo la consistencia del sistema distribuido.

Parte C — Tolerancia a fallos (peso orientativo 15%)

- Los nodos intercambian latidos (heartbeats) periódicos.
- Si un nodo deja de responder, los demás lo marcan como caído dentro de un tiempo acotado, registran el evento y el sistema continúa operando con los nodos restantes (gestión de fallo parcial).

Node.java

```

long elapsed =
    currentTime -
        status.getLastHeartbeat();

if (elapsed > 10000 &&
    status.isActive()) {

    status.setActive(false);

    System.out.println(
        "Nodo N" +
        entry.getKey() +
        " marcado como CAIDO"
    );
}

```

Este fragmento, ubicado en el método `startFailureDetector()` de `Node.java`, verifica el tiempo transcurrido desde el último heartbeat recibido por un nodo. Si han pasado más de 10 segundos sin comunicación y el nodo estaba marcado como activo, se considera que ha fallado y se actualiza su estado a inactivo para que el sistema pueda reaccionar ante la pérdida de disponibilidad.

Evidencias:

Durante la prueba se detuvo el nodo N3 y los demás nodos detectaron automáticamente su ausencia.

```

Run  N1 x N2 x N3 x Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N1 escuchando en puerto 5001
Coordinador actual: N3
[Lamport=1] Nodo N1 recibió operación: Movimiento_001
[Lamport=2] Nodo N1 recibió operación: Movimiento_001
Nodo N3 marcado como CAIDO
Coordinador caído → iniciando elección Bully...
Nuevo coordinador: N2
[Lamport=3] Nodo N1 recibió operación: Movimiento_001
No se pudo replicar a N3

```

gistroDistribuido > src > main > java > ec > edu > uteq > distribuidas > client > Client

Imagen 8. Continuidad de operación en el nodo N1 tras un fallo parcial. Se observa que el nodo continúa recibiendo y procesando operaciones después de detectar la caída de una de las réplicas, manteniendo el funcionamiento normal del sistema distribuido.

```

Run  N1 x N2 x N3 x Client x
C:\Users\ASUS\jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Nodo N2 escuchando en puerto 5002
Coordinador actual: N3
[Lamport=2] N2 recibió réplica: Movimiento_001
[Lamport=3] N2 recibió réplica: Movimiento_001
Nodo N3 marcado como CAIDO
Coordinador caído → iniciando elección Bully...
Nuevo coordinador: N2
[Lamport=4] N2 recibió réplica: Movimiento_001

```

gistroDistribuido > src > main > java > ec > edu > uteq > distribuidas > client > Client

Imagen 9. Procesamiento de operaciones en el nodo N2 después de la detección de un fallo. Se evidencia que el nodo permanece operativo y continúa participando en la replicación de información, contribuyendo a la disponibilidad del servicio pese a la ausencia de una réplica.

Parte D — Coordinación (peso orientativo 10%)

- Implemente la elección de coordinador con el algoritmo Bully: el nodo activo de mayor identificador actúa como coordinador.
- Ante la caída del coordinador, el sistema debe disparar y completar una nueva elección sin intervención manual.

BullyElection.java

```

package ec.edu.uteq.distribuidas.consensus;

import ec.edu.uteq.distribuidas.model.NodeStatus;

import java.util.Map;

public class BullyElection {

    public static int electLeader(

```

```

        int myId,
        Map<Integer, NodeStatus> nodes
    ) {

        int leader = myId;

        for (Map.Entry<Integer, NodeStatus> entry
             : nodes.entrySet()) {

            if (entry.getValue().isActive()) {

                if (entry.getKey() > leader) {
                    leader = entry.getKey();
                }
            }
        }

        Coordinator.setCoordinatorId(leader);

        return leader;
    }
}

```

Esta clase implementa el algoritmo Bully, utilizado para elegir un nuevo coordinador dentro del sistema distribuido cuando el coordinador actual deja de estar disponible. En conjunto con Node.java, revisa los nodos que permanecen activos y asigna la coordinación al nodo con el identificador más alto, permitiendo que el sistema continúe funcionando de manera organizada y sin intervención manual.

Evidencias:

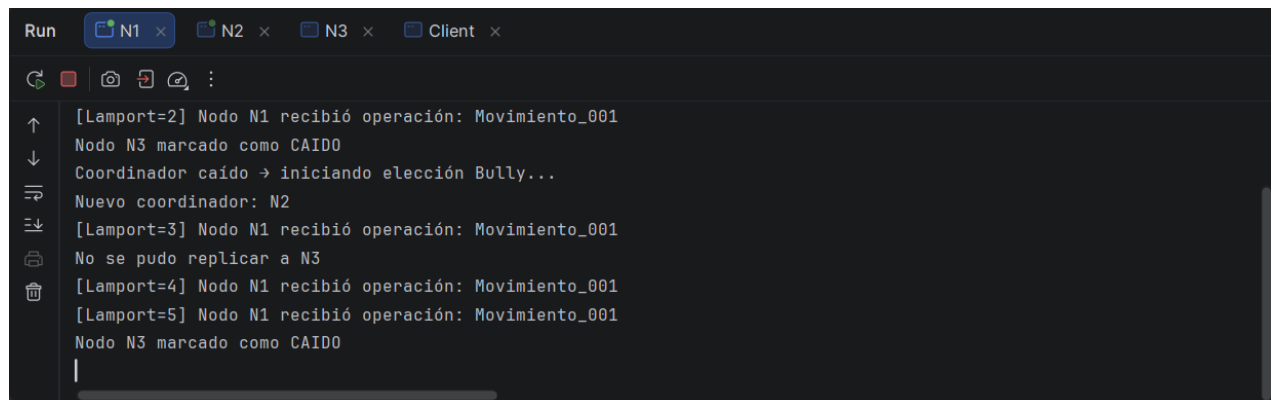


Imagen 10. Detección de la caída del coordinador y elección de un nuevo líder en el nodo N1. Se observa que, tras la detención del nodo N3, identificado previamente como coordinador, el sistema detecta su indisponibilidad e inicia automáticamente el proceso de elección. Como resultado, el nodo N2 es seleccionado como el nuevo coordinador al ser el nodo activo con el identificador más alto disponible.

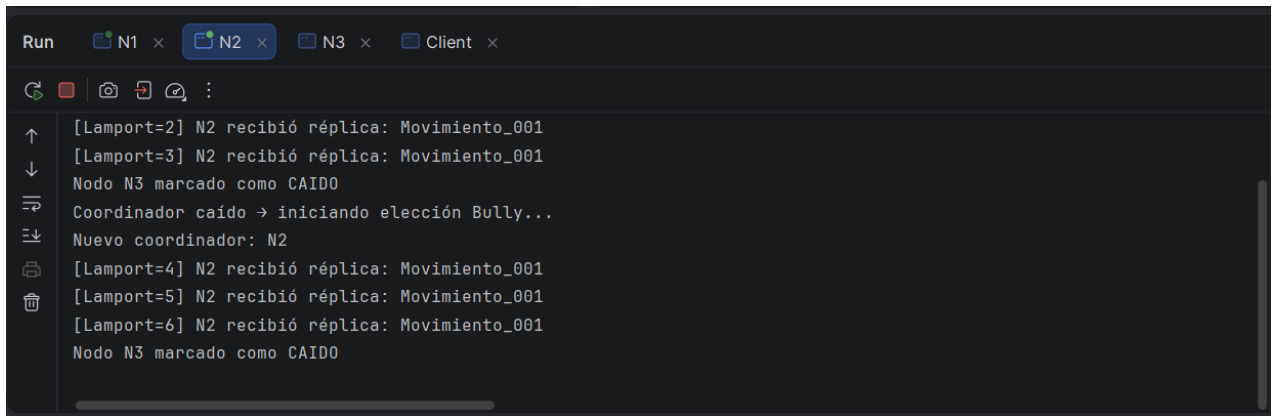


Imagen 11. Reelección automática del coordinador tras la detención del nodo N3. Se evidencia que, una vez detenido el nodo N3, el nodo N2 participa en el proceso de elección Bully y asume el rol de nuevo coordinador del sistema, garantizando la continuidad de las tareas de coordinación sin necesidad de intervención manual.

Parte E — Seguridad (peso orientativo 10%)

- El cliente debe autenticarse mediante un token antes de que el nodo acepte sus operaciones; las solicitudes sin token válido se rechazan.

TokenValidator.java

```

package ec.edu.uteq.distribuidas.core;

public class TokenValidator {

    private static final String VALID_TOKEN = "TOKEN123";

    public static boolean validate(String token) {
        return VALID_TOKEN.equals(token);
    }
}

```

Esta clase se encarga de verificar la validez del token enviado por el cliente antes de procesar una solicitud. En la interacción entre Client.java y Node.java, permite que el nodo compruebe que la petición proviene de una fuente autorizada, aceptando únicamente aquellas operaciones que incluyan el token configurado como válido.

Evidencias:

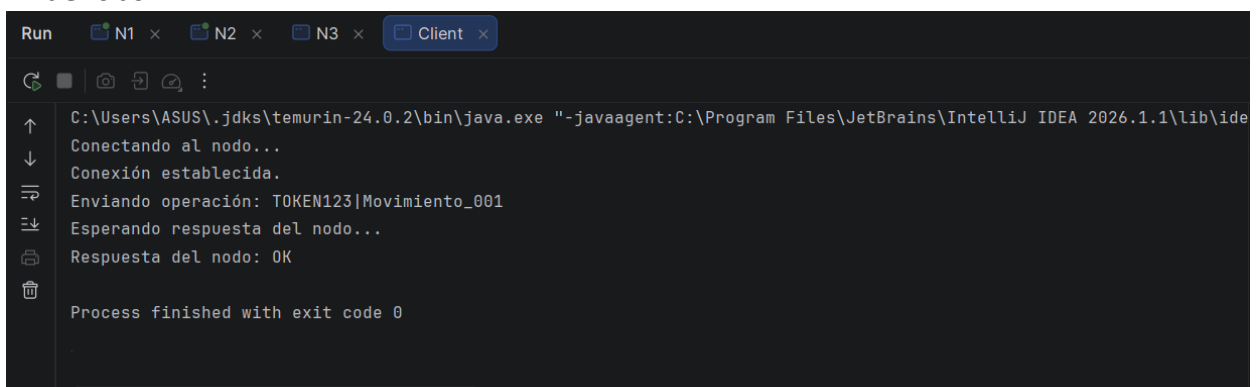


Imagen 12. Autenticación exitosa mediante un token válido. Se observa que el cliente establece la conexión

con el nodo y envía una operación acompañada de un token autorizado. Como resultado, la solicitud es aceptada y el nodo devuelve una respuesta de confirmación satisfactoria.

```

Run  N1 x  N2 x  N3 x  Client x
C:\Users\ASUS\.jdk\temurin-24.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\ide
Conectando al nodo...
Conexión establecida.
Enviando operación: TOKEN_MAL0|Movimiento_001
Esperando respuesta del nodo...
Respuesta del nodo: ERROR: TOKEN INVALIDO
Process finished with exit code 0
  
```

Imagen 13. Rechazo de una solicitud con un token no autorizado. Se evidencia que el sistema valida el token recibido antes de procesar la operación. Al detectarse un token inválido, la solicitud es rechazada y se informa al cliente mediante un mensaje de error, fortaleciendo el control de acceso al servicio distribuido.

4. Componente de análisis y defensa técnica

Al finalizar la implementación, el estudiante registrará sus respuestas en un archivo ANALISIS.md y las defenderá oralmente ante el docente. Las preguntas conectan el prototipo con la teoría de la unidad:

1. **Ante una interrupción de comunicación entre dos nodos, ¿qué propiedad del teorema CAP privilegia su implementación y por qué? Justifíquelo con el comportamiento observado en su prototipo.**

La implementación desarrollada privilegia principalmente la disponibilidad (Availability) dentro del teorema CAP. Cuando uno de los nodos deja de responder o existe una interrupción de comunicación, los nodos restantes continúan procesando operaciones y atendiendo solicitudes de los clientes. Durante las pruebas realizadas, al detener el nodo N3, los nodos N1 y N2 siguieron operando con normalidad, intercambiando información y manteniendo el servicio disponible. Aunque temporalmente podría existir una diferencia entre las réplicas debido a la ausencia del nodo caído, el sistema prioriza continuar funcionando antes que detener completamente las operaciones.

2. **¿Qué falacias de la computación distribuida tuvo que considerar al delimitar los mensajes y al definir los tiempos de espera de los latidos?**

Durante el desarrollo fue necesario considerar varias falacias de la computación distribuida. En primer lugar, no se asumió que la red fuera completamente confiable, por lo que se implementó una delimitación adecuada de mensajes mediante `writeUTF()` y `readUTF()`, evitando pérdidas o mezclas de información. También se consideró que la latencia no es constante, razón por la cual se estableció un tiempo de espera para los heartbeats antes de declarar un nodo como caído. Además, se tuvo presente que los recursos de red no son infinitos y que siempre existe la posibilidad de fallos de comunicación, por lo que el sistema incorpora mecanismos de detección y manejo de fallos parciales.

3. **¿Qué tipos de transparencia (ubicación, acceso, fallos, replicación) ofrece o no ofrece su solución? Argumente cada caso.**

La solución ofrece parcialmente varios tipos de transparencia. En cuanto a la transparencia de acceso, el cliente interactúa con el sistema mediante el mismo mecanismo de comunicación, sin necesidad de conocer detalles internos de la implementación. Respecto a la transparencia de replicación, esta se encuentra presente debido a que las operaciones son replicadas automáticamente entre los nodos sin intervención del cliente.

Por otro lado, la transparencia de fallos es parcial, ya que el sistema detecta la caída de nodos y continúa operando con las réplicas activas; sin embargo, los eventos de fallo son visibles en los registros de ejecución. Finalmente, la transparencia de ubicación no se encuentra completamente implementada, puesto que las direcciones y puertos de los nodos están definidos explícitamente dentro de la configuración del sistema.

4. **Proponga un acuerdo de nivel de servicio (SLA) de disponibilidad para este sistema y calcule el tiempo de inactividad anual admisible que implicaría.**

Para este prototipo se propone un SLA de disponibilidad del 99,9 %, ya que representa un nivel adecuado para un sistema distribuido que incorpora mecanismos básicos de tolerancia a fallos y continuidad del servicio.

Para calcular el tiempo máximo de inactividad permitido durante un año, se considera que un año tiene:

- 365 días
- 24 horas por día
- 60 minutos por hora

Por lo tanto:

$$365 \times 24 \times 60 = 525.600 \text{ minutos por año}$$

Dado que el SLA permite una indisponibilidad del 0,1 %, se realiza el siguiente cálculo:

$$525.600 \times 0,001 = 525,6 \text{ minutos}$$

Esto equivale aproximadamente a:

$$525,6 \text{ minutos} \approx 8 \text{ horas y } 46 \text{ minutos por año}$$

En consecuencia, con un SLA de disponibilidad del 99,9 %, el sistema podría permanecer fuera de servicio alrededor de 8 horas y 46 minutos al año sin incumplir el acuerdo de nivel de servicio establecido.

5. **Si reemplazara el algoritmo Bully por un consenso tipo Raft, ¿qué ganaría y qué costo introduciría?**

Si el algoritmo Bully fuera reemplazado por un mecanismo de consenso como Raft, se obtendría una mayor robustez en la elección de líderes y en la consistencia de la información compartida entre los nodos. Raft permite gestionar de manera más segura los cambios de liderazgo y reduce la posibilidad de inconsistencias durante fallos o recuperaciones de nodos.

Sin embargo, esta mejora también introduciría un mayor nivel de complejidad en la implementación. Sería necesario gestionar términos, votaciones, confirmaciones y replicación de registros mediante consenso, incrementando el intercambio de mensajes entre nodos y el consumo de recursos. En consecuencia, aunque Raft proporciona una solución más completa para sistemas distribuidos de mayor escala, el algoritmo Bully resulta más sencillo y adecuado para el alcance del prototipo desarrollado.

5. Entregables y condiciones

- **Repositorio:** Repositorio Git con el código fuente y un README que indique cómo compilar y ejecutar el prototipo.
- **Análisis:** Archivo ANALISIS.md con las respuestas del componente de análisis.
- **Demostración:** Demostración en vivo: levantar los tres nodos, enviar operaciones, mostrar el orden

total de eventos y provocar la caída de un nodo (incluido el coordinador).

- **Condición:** El prototipo debe compilar y ejecutarse siguiendo las instrucciones del README. Un proyecto que no ejecuta no puede acreditar los criterios funcionales.

6. Restricciones técnicas

- Lenguaje y stack: Java 21 + Spring Boot 3.x, o Python 3.11+. Comunicación mediante sockets TCP o gRPC.
- Ejecución local (varios puertos) o en contenedores Docker; IDE de referencia: IntelliJ IDEA.
- El uso de asistentes de IA y de material de consulta se rige por la política indicada por el docente para esta evaluación. En la defensa, el estudiante debe poder explicar y justificar cada parte de su código sin leerlo textualmente.

7. Rúbrica de evaluación

Cada criterio se valora en cuatro niveles: Excelente (4), Bueno (3), Regular (2) y Deficiente (1). Un criterio no entregado, con plagio verificado o que no corresponde recibe 0. La calificación final se obtiene como la suma ponderada de los niveles alcanzados, según la fórmula:

$$\text{Nota (sobre 10)} = [\sum (\text{peso}_i \times \text{nivel}_i) \div 4] \times 10$$

Donde los pesos se expresan como fracción (sumando 1,0) y el nivel es el valor 1 a 4 alcanzado en cada criterio. Si todos los criterios alcanzan el nivel Excelente, la nota es 10.

Rúbrica de evaluación — Examen Práctico Unidad 1

Criterio	Peso	Excelente (4)	Bueno (3)	Regular (2)	Deficiente (1)
C1. Comunicación entre procesos y delimitación de mensajes Peso: 20%	20%	Cliente y nodos se comunican por sockets TCP o gRPC; los mensajes se delimitan correctamente y nunca se solapan ni truncan; maneja varias operaciones seguidas sin pérdida.	La comunicación funciona y delimita los mensajes, con fallos menores ante concurrencia o casos límite.	Funciona en el caso simple pero no delimita los mensajes (se solapan/truncan) o solo atiende a un cliente.	La comunicación es inestable o no se establece de forma fiable.
C2. Relojes lógicos de Lamport y orden total Peso: 20%	20%	Aplica correctamente las reglas (incremento local y max+1 al recibir) y produce un log con orden total consistente, con desempate por ID; el orden respeta la causalidad.	Reglas correctas; el orden total funciona salvo en empates o detalles menores.	Implementa el contador pero aplica mal una regla (p. ej. no toma el máximo al recibir) o no genera orden total.	Relojes ausentes o incorrectos; no hay ordenamiento de eventos.
C3. Tolerancia a fallos: heartbeat y continuidad Peso: 15%	15%	Los nodos intercambian latidos; la caída se detecta en tiempo acotado, se registra y el sistema sigue operando con los nodos restantes.	Detecta la caída y continúa, con detección lenta o registro incompleto.	Detecta la caída pero el sistema se degrada de forma incorrecta o inestable.	No hay detección de fallos; la caída de un nodo detiene el sistema.
C4. Coordinación: elección de líder (Bully) Peso: 10%	10%	Implementa Bully correctamente; elige al nodo activo de mayor ID y, ante la caída del líder, dispara y completa una nueva elección sin intervención manual.	La elección funciona, pero la reelección tras la caída es parcial o requiere ajustes.	Designa un coordinador con criterio fijo, sin elección dinámica real.	No hay coordinación o el coordinador es estático/codificado a mano.
C5. Seguridad: autenticación del cliente Peso: 10%	10%	El nodo rechaza operaciones sin token válido y acepta solo clientes autenticados; el manejo del token es coherente.	Autentica correctamente, con detalles menores en el manejo del token.	Verifica el token de forma superficial o fácilmente evadible.	No hay autenticación; cualquier cliente es aceptado.
C6. Calidad del código, ejecución y repositorio Peso: 10%	10%	Compila/ejecuta siguiendo el README; código organizado y legible; instrucciones claras; repositorio con historial de commits.	Ejecuta con ajustes menores; organización aceptable.	Ejecuta con dificultad o instrucciones incompletas; código desordenado.	No compila/ejecuta o no hay repositorio utilizable.
C7. Análisis y defensa técnica Peso: 15%	15%	Responde con precisión las preguntas, justificando el compromiso CAP, las falacias, transparencias y un SLA razonable; defiende su código sin leerlo y razona ante preguntas inesperadas.	Análisis correcto en lo esencial con alguna imprecisión; defensa sólida.	Análisis superficial o parcialmente erróneo; defensa dubitativa.	No conecta la práctica con la teoría; no defiende su solución.

Nota: “0 puntos” se asigna a un criterio no entregado, con plagio verificado o que no corresponde al criterio. La calificación final aplica la fórmula ponderada indicada en la sección 7.